

Bachelor AI - MLOps

04 MLOps DEFINITION AND CHALLENGES

Tobias Mérinat

`tobias.merinat@hslu.ch`

HSLU Hochschule
Luzern

DEPARTMENT OF INFORMATION TECHNOLOGY
Lucerne University of Applied Sciences and Arts
6343 Rotkreuz, Switzerland

September 17/24 and October 15 2026



Elon Musk  

@elonmusk · [Follow](#)

Prototypes are easy, volume production is hard, positive cash flow is excruciating

9:40 PM · Jan 14, 2021

The Root Causes of Failure for Artificial Intelligence Projects and How They Can Succeed

Avoiding the Anti-Patterns of AI

[James Ryseff](#), [Brandon De Bruhl](#), [Sydney J. Newberry](#).

- 1 stakeholders often misunderstand what problem needs to be solved using AI
- 2 many AI projects fail because the organization lacks the necessary data to adequately train an effective AI model
- 3 in some cases, AI projects fail because the organization focuses more on using the latest and greatest technology than on solving real problems for their intended users.
- 4 **organizations might not have adequate infrastructure to manage their data and deploy completed AI models, which increases the likelihood of project failure.**
- 5 in some cases, AI projects fail because the technology is applied to problems that are too difficult for AI to solve.

Hidden Technical Debt in Machine Learning Systems

D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips
{dsculley, gholt, dgg, edavydov, toddphillips}@google.com
Google, Inc.

ML systems have a special capacity for incurring technical debt, because they have all of the maintenance problems of traditional code plus an additional set of ML-specific issues. This debt may be difficult to detect because it exists at the system level rather than the code level. Traditional abstractions and boundaries may be subtly corrupted or invalidated by the fact that data influences ML system behavior. Typical methods for paying down code level technical debt are not sufficient to address ML-specific technical debt at the system level.

AI & Machine Learning

Key requirements for an MLOps foundation

September 1, 2020

Craig Wiley

Director of Product Management, Cloud AI and Industry Solutions

... They have all of the maintenance problems of traditional code plus an additional set of ML-specific issues: ML systems have unique hardware and software dependencies, require testing and validation of data as well as code, and as the world changes around us deployed ML models degrade over time. Moreover, ML systems underperform without throwing errors, making identifying and resolving issues especially challenging. ...

*The existing mantra is that MLOps is about automating continuous integration (CI), continuous delivery (CD), and continuous training (CT) for ML systems. But that is too abstract for many developers. MLOps is really about **continual development of ML-enabled products that evolve over time**. The available input **data (features) changes over time**, the target you are trying to predict changes over time. You need to make changes to the source code, and you want to ensure that any changes you make do not break your ML system or degrade its performance. And you want to accelerate the time required to make those changes and test before those changes are automatically deployed to production.*

- DevOps -> MLOps
- DevOps is a foundation for the successful and efficient operation of software
- MLOps >> DevOps
- MLOps began as a set of best practices
- and has evolved towards complete lifecycle management for Machine Learning
- MLOps connects Data Science, Software Engineering, Data Engineering, and Operations
- MLOps encompasses a development culture for the end-to-end conception, implementation, monitoring, deployment, and scalability of software with a Machine Learning component.

- CI/CD Automation
- Workflow Orchestration
- Reproducibility
- Versioning of Data, Models, and Code
- Continuous Model Training
- Continuous Evaluation of Metrics and Data
- Tracking and Logging of Metadata
- Feedback Loops

Challenges (Batch)

- Starting point: Trained model is available
- Deployment as Batch Pipeline
 - The easy part
 - Read data
 - Calculate features
 - No time pressure
 - Generate and store predictions
 - But what if
 - Feature calculation was done with pandas and scikit-learn, but in production has to run on an **Apache Spark** cluster?
 - The calculated features must be stored in order to make them reusable, and joined with labels that arrive later?
 - No labels arrive, and model degradation still needs to be detected?

- Deployment in an App

- The easy part

- Simple feature calculation with request data
 - Load model, predict, use result

- But what if

- Feature calculation was done with pandas and scikit-learn, but the app is written in typeScript?
 - Feature calculation takes too much time to be done in realtime?
 - The model also requires features that don't come with the request?

Challenges (Streaming)

- When realtime feature calculation doesn't scale. . .
- . . . and batch features are not current enough,
- we need Stream Processing
- Joining of features over time that arrive from sources with different frequencies
- Time-Travel for Retraining
 - Features can come from different sources
 - and arrive at different frequencies
 - Labels arrive later in the process

- A trained model is a snapshot in time
- If conditions change, model performance can degrade
- Continuous monitoring is necessary at all stages
 - Raw data
 - Features
 - Predictions (if possible)
 - Metrics (if possible)
 - Operational metrics

High-Level Requirements

- **Reliability:** Even under unfavorable conditions (e.g., a hardware crash or corrupt data), the system continues to produce correct predictions or at least indicates that this is no longer the case.
- **Scalability:** Machine Learning systems can grow in various dimensions: same task but more complex (and thus hopefully more accurate) model, more traffic, more models running in parallel, more complex feature transformations, stricter requirements for latency and throughput.
- **Maintainability:** This includes the ability to identify, analyze, and fix problems; simple and ideally automated deployment, governance and reproducibility, as well as generally easy and efficient collaboration among all involved teams.
- **Adaptability:** Machine Learning systems consist of code and data. Data can change quickly depending on the use case, which is why Machine Learning systems must be efficiently and easily adaptable. This naturally also requires high maintainability.

Challenges from Easy to Hard

■ Level 1

- Experiment Tracking and Model Registry
- Model Deployment
- Batch Prediction

■ Level 2

- Streaming Prediction
- Drift Detection
- Streaming Feature Engineering

■ Level 3

- Feature Management
- Feature Stores

■ Level 4

- Continual Learning

There is a thing called the **PoC to Production Gap** and it is real